5d. Vector Creation and Initialization

Martin Alfaro

PhD in Economics

Code Script

This section's scripts are available **here**, under the name `allCode.jl`. They've been tested under Julia 1.11.8.

INTRODUCTION

We continue presenting preliminary concepts for introducing the concept of mutation. The previous section distinguished between the use of `=` for assignments and mutations. Now, we'll deal with approaches to creating vectors.

Our presentation starts by outlining the process of initializing vectors, where memory is reserved without assigning initial values. We'll then discuss how to create vectors filled with predefined values such as zeros or ones. Finally, we show how to concatenate multiple vectors into new ones through the `vcat` function.

INITIALIZING VECTORS

Creating an array involves two steps: reserving memory for holding its content and assigning initial values to its elements. When you don't intend to populate the array with values right away, it's more efficient to perform only the allocation step. This means reserving memory space, but without setting any initial values.

Technically, initializing an array entails creating an array filled with `undef` values. These values represent arbitrary content in memory at the moment of allocation. Importantly, while `undef` displays concrete numbers when you output the array's content, they're meaningless and vary every time you initialize a new array.

There are two methods for creating vectors with `undef` values. The first one requires you to explicitly specify the type and length of the array, which is accomplished via `Vector{<elements' type>}(undef, <length>)`. The second approach is based on the function `similar(x)`, which creates a vector with the same type and dimensions as an existing vector `x`.

'undef' Values | 'similar'

'UNDEF' VALUES

```
x_length = 3

x            = Vector{Int64}(undef, x_length)   # `x` can hold `Int64` values, and is initialized with 3 undefined elements
```

```
julia> x
3-element Vector{Int64}:
  1
 46
 42
```

The example demonstrates that `undef` values don't follow any particular pattern. Moreover, these values vary in each execution, as they reflect any content held in RAM at the moment of allocation. In fact, a more descriptive way to call `undef` values would be **uninitialized values**.

CREATING VECTORS WITH GIVEN VALUES

In the following, we present several approaches to creating arrays filled with predefined values.

VECTORS COMPRISING A RANGE

To generate a sequence of values through ranges, we need to employ the expression `collect(<range>)`. Recall that the syntax for defining ranges is `<start>: <steps>: <stop>`, where `<steps>` specifies the increment between consecutive values.

The function `collect` is necessary, since ranges describe the values to be generated, without materializing them. This behavior is consistent with a broader concept known as lazy operations, which will be presented later on this book. [1]

A Vector Filled Through a Sequence

A VECTOR FILLED THROUGH A SEQUENCE

```
some_range = 2:5

x          = collect(some_range)
```

Output in REPL

```
julia> x
4-element Vector{Int64}:
 2
 3
 4
 5
```

When a range is created, `<steps>` dictates the number of elements to be generated. Considering this, Alternatively, we can specify the number of elements to be stored through the syntax `<start> : 1/<number of elements> : <end>`. A more direct way is via the `range` function, whose syntax is `range(<start>, <end>, <number of elements>)`.

The following code snippet demonstrates the use of the `range` function, by generating five evenly spaced elements between 0 and 1.

Range | Range (Keyword Arguments) | Range (Keyword Arguments)

RANGE

```
x = range(0, 1, 5)
```

Output in REPL

```
julia> x
0.0:0.25:1.0
```

VECTORS WITH SPECIFIC VALUES REPEATED

We can also build vectors of a given length where every element is the same value. Two common examples are `zeros` and `ones`, which produce vectors filled with zeros and ones, respectively. By default, both functions create vectors whose elements have type `Float64`. You can override this behavior by providing the desired element type as the first argument.

Zeros | Zeros with Type | Ones | Ones with Type

ZEROS

```
length_vector = 3

x             = zeros(length_vector)
```

Output in REPL

```
julia> x
3-element Vector{Float64}:
 0.0
 0.0
 0.0
```

To create vectors with Boolean values, Julia provides two convenient functions called `trues` and `falses`.

Trues | Falses

TRUES

```
length_vector = 3

x             = trues(length_vector)
```

Output in REPL

```
julia> x
3-element BitVector:
 1
 1
 1
```

VECTORS WITH AN ARBITRARY VALUE REPEATED

More generally, we can define vectors whose elements share the same specified value. This is achieved by the `fill` function.

Fill (scalar) | Fill (vector) | Fill (vector - single element)

FILL (SCALAR)

```
length_vector  = 3
filling_object = 1

x              = fill(filling_object, length_vector)
```

Output in REPL

```
julia> x
3-element Vector{Int64}:
 1
 1
 1
```

A VECTOR WITH CONCATENATED ELEMENTS OF VECTORS

There are several ways to construct a vector `z` that combines all elements of two vectors `x` and `y`. A straightforward method is using `z = [x; y]`. While this works well for concatenating a small number of vectors, it becomes cumbersome when many vectors must be merged. Moreover, it's not viable when the number of vectors is unknown in advance.

In such cases, we can employ the `vcat` function. This concatenates all of its arguments into a single vector. When paired with the splat operator `...`, it can also operate on a single argument containing multiple vectors.

vcat | vcat and Splatting Operator

VCAT

```
x = [3,4,5]
y = [6,7,8]


z = vcat(x,y)
```

Output in REPL

```
julia> x
6-element Vector{Int64}:
 3
 4
 ⋮
 7
 8
```

VECTORS WITH ELEMENTS OF AN OBJECT REPEATED

Closely related to `fill` is the `repeat` function. This uses a *collection* as an input, concatenating their elements repeatedly a given number of times. The `repeat` function **necessarily requires an array as its input**, throwing an error if a scalar is passed.

Repeat (vector) | Repeat (vector - single element) | Repeat (scalar - not allowed)

REPEAT (VECTOR)

```
nr_repetitions     = 3
elements_to_repeat = [1,2]

x                  = repeat(elements_to_repeat, nr_repetitions)
```

Output in REPL

```
julia> x
6-element Vector{Int64}:
 1
 2
 ⋮
 1
 2
```

Note that the behavior of `repeat` differs from `fill` function, since the latter repeats the same object without concatenating its elements. In fact, the output of `repeat` is the same as creating a vector with `fill` and then using `vcat` to concatenate its elements.

'repeat' With Vector Input | 'fill' With Vector Input | 'fill' + 'vcat'

'REPEAT' WITH VECTOR INPUT

```
nr_repetitions     = 3
elements_to_repeat = [1,2]

x                  = repeat(filling_object, nr_repetitions)
```

Output in REPL

```
julia> x
3-element Vector{Int64}:
 1
 1
 1
```

ADDING, REMOVING, AND REPLACING ELEMENTS (*OPTIONAL*)

⚠️ **Warning!**

This subsection requires knowledge of a few **concepts we haven't discussed yet**. As such, it's marked as optional.

One such concept is that of **in-place functions**, identified by the symbol `!` appended to the function's name. The symbol is simply a convention chosen by developers to indicate that the function modifies at least one of its arguments. A detailed discussion of in-place functions will be provided later.

Another concept introduced is that of **pairs**, which will be thoroughly examined in a future section too. For the purposes of this subsection, it's sufficient to know that pairs are written in the form `a => b`. In our

applications, a will represent a given value and b denotes its corresponding replacement value.

Next, we demonstrate how to add, remove, and replace elements within a vector. Below, we begin by presenting methods for adding a single element.

Add Scalar At End | Add One Element At Start | Add One Element at Specific Index | Add Multiple Elements At End

ADD SCALAR AT END

```
x                = [3,4,5]
element_to_insert = 0


push!(x, element_to_insert)                # add 0 as last element - faster
```

Output in REPL

```
julia> x
4-element Vector{Int64}:
 3
 4
 5
 0
```

The function `push!` is particularly helpful to collect outputs in a vector. Since it doesn't require prior knowledge of how many elements will be stored, the vector can grow dynamically as new results are added. Notice that adding elements at the end with `push!` is generally faster than inserting them at the beginning with `pushfirst!`.

Analogous functions exist to remove elements, as shown below.

Remove Last Element | Remove First Element | Remove One Element at Index | Remove Multiple Elements at Index

REMOVE LAST ELEMENT

```
x                = [5,6,7]



pop!(x)                            # delete last element
```

Output in REPL

```
julia> x
2-element Vector{Int64}:
 5
 6
```

By analogy with the behavior of `deleteat!`, it's also possible to specify which elements should be retained.

Keep Single Element | Keep Multiple Elements

KEEP SINGLE ELEMENT

```
x               = [5,6,7]
index_to_keep   = 2

keepat!(x, index_to_keep)
```

Output in REPL

```
julia> x
1-element Vector{Int64}:
 6
```

Finally, specific values can be replaced with new ones. This can be done by either creating a new copy using `replace` or by updating the original vector with `replace!`.

Both functions make use of pairs `a => b`, where `a` denotes a given value and `b` specifies its replacement. Note that these functions perform substitutions based on values, not on indices.

Replace Single Value (update) | Replace Multiple Values (update) | Replace Single Value (copy) | Replace Multiple Values (copy)

REPLACE SINGLE VALUE (UPDATE)

```
x = [3,3,5]

replace!(x, 3 => 0)              # in-place (it updates x)
```

Output in REPL

```
julia> x
3-element Vector{Int64}:
 0
 0
 5
```

Previous Section Next Section

FOOTNOTES

[1] Lazy operations specify how a computation should proceed, but postpone producing concrete results until they're explicitly required (either because you request them or because another computation depends on them.) They become especially valuable when combined with other operations, since this fusion may eliminate the need to store intermediate results altogether.